

Article

BACH: A Tool for Analyzing Blockchain Transactions Using Address Clustering Heuristics

Michele Caringella ¹, Francesco Violante ^{1,*}, Francesco De Lucci ¹, Stefano Galantucci ²  and Matteo Costantini ²
¹ Italpaghe S.r.l., Viale Paolo Borsellino e Giovanni Falcone, 17, 70125 Bari, Italy

² Department of Computer Science, University of Bari Aldo Moro, 70125 Bari, Italy; stefano.galantucci@uniba.it (S.G.); matteo.costantini00@gmail.com (M.C.)

* Correspondence: rs@progetto-across.com

Abstract: Cryptocurrencies have now become an emerging blockchain-based payment technology; among them, bitcoin is the best known and most widely used. Users on these networks are pseudo-anonymous, meaning that while all transactions from an address are transparent and searchable by anyone, the users' true identities are not directly revealed; to preserve their privacy, users often use many different addresses. In recent years, some studies have been conducted regarding analyzing clusters of bitcoin addresses that, according to certain heuristics, belong to the same entity. This capability provides law enforcement with valuable information for investigating illegal activities involving cryptocurrencies. Clustering methods that rely on a single heuristic often fail to accurately and comprehensively cluster multiple addresses. This paper proposes *Bitcoin Address Clustering based on multiple Heuristics* (BACH): a tool that uses three different clustering heuristics to identify clusters of bitcoin addresses, which are displayed through a three-dimensional graph. The results lead to several analyses, including a comparative evaluation of WalletExplorer, which is a similar address clustering tool. BACH introduces the innovative feature of visualizing the internal structure of clusters in a graphical format. The study also shows how the combined use of different heuristics provides better results and more complete clusters than those obtained from their individual use.

Keywords: bitcoin; address clustering; blockchain; anonymity; heuristics



Citation: Caringella, M.; Violante, F.; De Lucci, F.; Galantucci, S.; Costantini, M. BACH: A Tool for Analyzing Blockchain Transactions Using Address Clustering Heuristics. *Information* **2024**, *15*, 589. <https://doi.org/10.3390/info15100589>

Academic Editor: Ge Yu

Received: 8 July 2024

Revised: 11 September 2024

Accepted: 16 September 2024

Published: 26 September 2024



Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Major electronic payment methods rely almost exclusively on financial institutions, which are trusted third parties, to process payments. Despite the effectiveness of these systems, they still suffer from inherent weaknesses derived from a trust-based model. Moreover, due to mediation disputes, totally irreversible transactions are not possible. The possibility of reversibility also comes with the need for trust, and a certain percentage of fraud is accepted as inevitable. On 31 October 2008, Satoshi Nakamoto, whose identity is still unknown, sent an article entitled “Bitcoin: an Electronic Peer-to-Peer Cash System” [1] to a mailing list of cryptography experts. bitcoin began as an electronic payment system based on cryptography rather than trust, allowing two counterparties to negotiate directly without needing a trusted third party. Cryptography makes transactions computationally irreversible, protecting sellers from fraud attempts. bitcoin solves the double-spending problem through the use of a peer-to-peer network. Transactions are grouped into blocks, which form a chain called a blockchain. Thanks to the proof-of-work mechanism, bitcoin guarantees the irreversibility of transactions: once registered in the public blockchain, transactions cannot be changed unless the nodes collaborating to attack the network control most of the computational power, as in the case of a 51% attack [2]. Identities in the bitcoin network are pseudo-anonymous. Pseudo-anonymity in the context of the bitcoin network refers to the degree of anonymity that users are afforded: although public, bitcoin transactions do not directly reveal the identities of the people involved, as the only information that can be traced back to users is bitcoin addresses linked to money movement.

These addresses act as pseudonyms, providing a level of confidentiality by masking the identity of the address owner. For this reason, transactions on the bitcoin network are said to be “*pseudo-anonymous*”. Unlike true anonymity, which has no link to personal identity, pseudo-anonymity implies that addresses can be traced back to real-world identities through specific analyses. For example, if a user purchases items that will be shipped by paying in bitcoin or converts their bitcoins to money in real-world accounts, such actions effectively break pseudo-anonymity, as they expose information that can be directly traced back to the person. This leads to the concern that the combination of scalable, irrevocable, and (pseudo-)anonymous payments is very attractive to criminals involved in fraudulent activities and money laundering. Over the years, several methods have been developed to break the pseudo-anonymity that bitcoin is supposed to provide [3–9] by identifying clusters of addresses. The address clustering problem is a problem that, by its very nature, does not have a precise solution, so heuristic techniques are generally applied [10], i.e., problem-solving strategies that, through the use of shortcuts, approximations, or greedy approaches, aim to find a good and fast solution when an exhaustive search for an exact solution is impractical or senseless. In the context of address clustering, heuristics use trends and actions seen in transactional data to identify clusters of addresses assumed to belong to the same entity, inferring links between addresses. Such techniques, in any case, run up against the issue of the bitcoin address controller. The controller of an address may know the corresponding private key, but this does not imply that it is the owner. For example, if a user buys bitcoins from an exchange such as Mt. Gox, his or her funds will be contained in an address generated by the exchange, which knows the corresponding private key and can conduct transactions with it. In this case, Mt. Gox is the controller of the generated address, even though the funds contained in it belong to the user. The study of clusters results in potentially huge groups of addresses, as the number of addresses controlled by a service such as Mt. Gox will be huge. Many studies have focused on a single analysis of clusters of major entities but without studying the complete graph of the entire network divided into groups. Blockchain analysis software based on these results has also been developed, such as BitConduite [11] and BitExtract [12]. The currently known and used tools allow various graphical visualizations of the bitcoin network [13] but do not graphically show the clusters of addresses obtained. On the other hand, tools that perform address clustering using heuristics only deal with identifying the set of addresses belonging to a cluster but do not detail the internal structure of the cluster, let alone offer a graphical visualization of them and the relationships between addresses in the cluster. Where graphical tools exist, they only show the flow of cash and transactions between addresses but do not perform analyses aimed at understanding clusters of addresses referable to the same entity.

To overcome such limits, this paper proposes a tool called *Bitcoin Address Clustering based on multiple Heuristics* (BACH) that can partition the entire bitcoin network into groups of addresses using variations of heuristics already known in the literature. BACH, unlike other tools, allows the visualization of the various clusters found in a 3D graph along with the relationships between the addresses of the same, thus allowing the overall structure of the clusters to be framed and any patterns within them to be identified. Moreover, the effectiveness of BACH comes from the fact that, unlike other known tools, it allows the simultaneous application of multiple heuristics rather than just one. The main theoretical contribution of this work is that it shows how the combined application of multiple heuristics for clustering addresses can bring the cluster structure to a more complete and more accurate aggregation compared to using one single heuristic. The approach greatly broadens the possibilities for detecting hidden correlations between addresses, new theoretical means for treating blockchain networks, and the behavior of actors in the network. Moreover, the three-dimensional visualization of the clusters proposed by BACH is a methodological improvement because it allows the assessment of the relationships between the variables inside the cluster to be made more intuitive and detailed. This allows the cluster to be understood and visualized as a graph composed of nodes and arcs.

The paper is organized as follows: Section 2 discusses related work, Section 3 details the heuristics that are used in BACH, Section 4 describes the operation of BACH and its components, Section 5 describes the experiments performed to evaluate the operation of BACH, and finally, the conclusions are given in Section 6.

2. Related Work

This section provides a focused selection of articles and cited works related to this study. Due to the interconnected nature of this topic, studies exploring associated areas such as malicious transaction analysis (e.g., CoinJoin transactions), taint analysis, and the use of blending services for money laundering are also reviewed. A search of various scientific databases (Scholar and Scopus) is conducted, with a focus on peer-reviewed articles and conference papers to provide a comprehensive view of the methods and challenges in the field. These papers provide a factual basis on the topic at hand, allowing us to choose only two main heuristics—multi-input address clustering and modified address clustering—which, according to our analysis, have a higher reliability index; thus we exclude less robust methods. In addition, a heuristic related to Coinbase transactions is used, for which the literature related to the structure of mining pools and how they distribute mining rewards has been studied. Several papers focus on the problem of address clustering using heuristics: an approach in which the structural information of given transactions is exploited to infer relationships between addresses. Androulaki et al. [14] defined the concept of a shadow address as a change address. From that idea, Meiklejohn et al. [5] identified a heuristic that can cluster such addresses; they also offer a more rigorous definition of heuristics based on multiple input addresses and change addresses for clustering and analyze the discrepancy between actual and potential anonymity in bitcoin. Another variant of the heuristics of change addresses was proposed by Ermilov et al. [15]; this variant is more restrictive than that in [5], as it only considers transactions that have two output addresses. This additional condition implies a much more accurate and certain identification of such change addresses; however, this strategy, on the other hand, results in a high number of false negatives; such a heuristic has not been chosen for BACH precisely because it aims much more at certainty about the case than at including even cases that are not perfectly certain but have a high probability index of falling within one of the techniques for pseudo-anonymity. Later, in the work of Zhang et al. [16], an improvement to the heuristic proposed by [5] was introduced, in which, in order to identify the change address, future transactions to the one currently being analyzed must also be analyzed; however, in this way, it becomes difficult to analyze the quality of the heuristic used for clustering. Finally, Zhao et al. [17] proposed a variant of such heuristics on the change address: this has been chosen to be implemented in our work, and it also considers the amounts transferred from each address during the transaction. Zheng et al. [18] proposed a heuristic based on coinbase address clustering, according to which all output addresses in a coinbase transaction are controlled by the same group of users. An approach to identify transactions belonging to mining pools was also discussed in this paper. However, given the operational structure of some mining pools, as described by Lewenberg et al. [19], this heuristic becomes less accurate. For this reason, a threshold was added to the heuristic on coinbase address clustering in BACH so that it is not applied to those transactions that reflect the patterns of some mining pools. Reid and Harrigan [20] studied anonymity and observed the extent to which the whole bitcoin graph can be divided into two independent DAGs (directed acyclic graphs). In the first DAG, they monitored the flow of bitcoins between different users. The second DAG represented transaction analysis over time, where transactions were represented as nodes. Directed edges connected source and target bitcoin addresses to model the flow from one transaction's output to another's input and thus formed a transaction chain. Other studies in this direction involve revisiting and analyzing the transaction graph from many perspectives. Ron and Shamir [21] contributed to this endeavor by devising a graph of the largest transactions in bitcoin and then breaking this graph down into a series of sub-graphs. They followed such an approach to discover

a number of easily identifiable patterns in the flow of bitcoin transactions. Spagnuolo et al. [22] presented a framework for forensic analysis of illegal bitcoin transactions, and Bitiodine, a graph analysis and automated investigation tool, was born. These studies are based on the analysis of the graph constructed from bitcoin transactions, where the source address is linked to the destination address. In the present study, however, a new approach is used, where the graph is not constructed based on transactions, but rather, we aim to build the graph of a cluster using clustering heuristics, allowing for the highlighting of internal relationships among the various addresses within a cluster and facilitating the identification of potential patterns.

3. Heuristics Employed

Address clustering techniques are useful for analyzing transactions occurring in the network. Once a cluster of addresses is found, off-chain information can be combined to trace the entity's identity related to the cluster. Multiple heuristics can be simultaneously used to improve the level of aggregation of the entities found. The following three heuristics were chosen:

- Multi-input address clustering: identifies groups of addresses by analyzing transactions for which multiple input addresses are used together, assuming they are controlled by the same entity.
- Change address clustering: identifies addresses that receive “change” from bitcoin transactions. When a transaction is made, if the input amount exceeds the output amount, the difference (change) is sent to a new address that is typically controlled by the sender.
- Coinbase address clustering: coinbase transactions, which are the first transactions in a block and reward miners, often direct outputs to specific addresses that are typically controlled by the same mining entity.

The first heuristic was chosen for its high accuracy, which is derived from the bitcoin protocol's requirement for private key signatures on each input. So it is safe to assume that all addresses in the inputs belong to a single user. For the heuristic on identifying change addresses, several variants exist, but the one proposed by Zhao et al. [17] was chosen due to its reliability, as it is one of the most comprehensive. Additionally, this heuristic, compared to other variants, has a higher aggregation rate. This allows it to identify fewer but larger clusters rather than numerous smaller ones. The final heuristic employed involves grouping the output addresses of a coinbase transaction. This is a minor heuristic but was implemented to enhance aggregation by including addresses that are correlated through the outputs of a coinbase transaction. This additional layer of clustering was chosen because it helps to provide a more comprehensive view by incorporating addresses that might otherwise be overlooked.

The three heuristics are disjointed from each other and provide three non-overlapping perspectives. They operate differently and, more importantly, aggregate by looking for combinations based on different logic: for this reason, their combination succeeds at generating clusters that include different ways of aggregation. Suppose a user uses multiple techniques to favor his or her own pseudo-anonymity and many addresses; single heuristics would be able to identify only subsets of those addresses. In that case, the combination of several heuristics, on the other hand, allows for the union of the different sets. Further details on the accuracy of the heuristic can be found in the following paragraphs.

3.1. Multi-Input Address Clustering

The first heuristic presented, which deals with clustering all input addresses to a transaction, is one of the most widely used heuristics for clustering. Androuraki et al. [14] studied transaction input addresses and developed this heuristic.

Definition 1. *If two (or more) addresses are inputs to the same transaction, they are controlled by the same user; that is, for each transaction t , each $pk \in \text{input}(t)$ is controlled by the same entity.*

The effects of this heuristic are transitive, i.e., if a transaction with addresses A and B as input is observed, and then another with addresses B and C as input, it can be inferred that addresses A , B , and C all belong to the same user. As introduced earlier, there may be potentially massive clusters due to services such as Mt. Gox that control a large number of addresses belonging to different users; despite this, they will still be clustered into one large cluster. In this case, these are not false positives since these services have access to users' private keys and control addresses. The multi-input clustering heuristic is widely used because it is highly accurate: to sign a transaction, the sender must know all the private keys of the corresponding public keys used for the various inputs. According to bitcoin's protocol, to transfer bitcoins from an address, the respective private key must be provided; that is, the transaction must be signed. For this reason, the public keys used as inputs are unlikely to be controlled by different users since they should know everyone else's private keys. Furthermore, the transaction is initialized by only one bitcoin client; therefore, all input addresses can be considered to belong to the same entity. The accuracy of this heuristic can reach 100% if the cases in which users use mixing services or CoinJoin transactions [23] to intentionally avoid being tracked through clustering are not considered. In these situations, the heuristic generates false positives, i.e., addresses not belonging to the same user are clustered together.

3.2. Change Address Clustering

The change address is used in a transaction to return any change to the sender. It is usually a new address generated by the client itself, i.e., the bitcoin wallet, during the creation of the transaction. To construct a transaction, the bitcoin wallet must refer to the currently available set of UTXOs (unspent transaction outputs): that is, the set of outputs that can be spent. In the bitcoin protocol, a UTXO is indivisible, so it must be used in its entirety. If the UTXO is greater than what the transaction's sender wants to spend, a new address will be created to receive the change (change address), to which the new UTXO will be sent. In the past, bitcoin wallets did not generate a new address to receive change but used the same input address as the sender. To improve pseudo-anonymity on the network, now new wallets automatically generate a new address for each transaction to be used as the change address. In the work by Androutaki et al. [14], in addition to introducing heuristics on multi-input clustering, a heuristic is also presented for identifying the change address of a transaction so that it can be added to the cluster of input addresses and improve the aggregation level. In the article, addresses automatically generated by bitcoin clients to receive change are called "shadow addresses". Over the years, many researchers have tried to improve this heuristic to minimize false positives and avoid constructing unreliable clusters. The proposed work uses a variation of this heuristic that was proposed by Zhao et al. [17], who presented a method that considers whether the output address is new or already in the blockchain and evaluates the amounts transferred in the transaction.

Definition 2. Let t be a bitcoin transaction, a an output address of t , A the set of all bitcoin addresses currently recorded on the blockchain, $Input(t)$ be the set of input addresses of transaction t , and $Output(t)$ be the set of output addresses of transaction t . If t is a transaction and a is an input/output address of t , let $value(t, a)$ be the amount of currency sent or received by address a for transaction t :

1. t is not a coinbase transaction;
2. $\exists ai \in Output(t), ai \neq a$;
3. $\forall an \in \{am | am \in Output(t), am \neq a\}, an \in A$;
4. $\exists an \in \{am | am \in Output(t), am \neq a\}, an \notin A$;
5. $\forall an \in \{am | am \in Output(t), am \neq a\}, value(t, a) < value(t, an)$;
6. $\exists ! ax \in Output(t), \forall an \in Input(t), value(t, ax) < value(t, an), ax = a$.

If address a satisfies conditions 1, 2, and 3 or 1, 2, 4, 5, and 6, then this is the change address of transaction t .

Thus, the conditions expressed in Definition 2 are as follows:

1. Transaction t is not a Coinbase transaction. Miners create this special type of transaction as a reward for successfully mining a new block. This transaction is unique because it is the only way new bitcoins are introduced into circulation.
2. There is at least one other output address ai of t that is different from a .
3. For all output addresses an of transaction t that are different from a , these addresses must already be recorded on the blockchain (belong to the set A of currently recorded addresses).
4. This point is an alternative to the previous one and states that there is at least one output address an of transaction t that is different from a and is not already recorded on the blockchain (does not belong to the set A).
5. The amount sent to address a is less than the amounts sent to all other output addresses of transaction t .
6. There is exactly one address a among the outputs of transaction t for which the amount sent is less than all the amounts of the inputs of transaction t .

If address a satisfies conditions 1, 2, and 3, then it is the only new address among all the outputs of t . In this case, address a is labeled as the change address and is added to the transaction input address cluster, as shown in Figure 1.

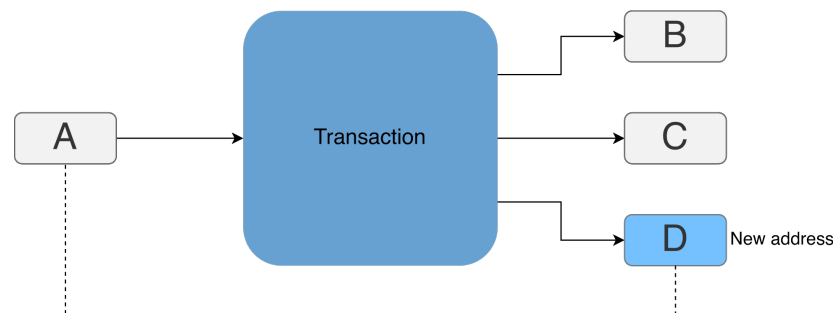


Figure 1. Change address example: output address D is the change address.

On the other hand, if address a satisfies conditions 1, 2, 4, 5, and 6, then this means that there is not a single new address in the output of t , and address a is the only one among them for which the amount is less than all the amounts of the inputs of t . Furthermore, address a is the address to which the least amount of value has been sent. In this case, address a is the change address, as shown in Figure 2.

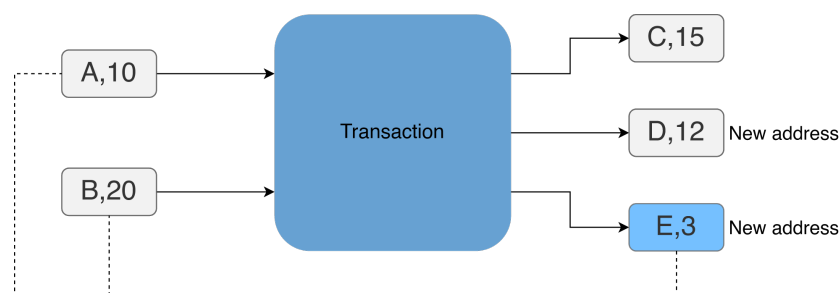


Figure 2. Change address example: output address E is the change address.

If no output address in a transaction verifies these conditions, no address will be marked as a change address. In addition, transactions for which an address in the outputs belongs to the transaction's inputs are not considered, since this address is usually just the change address and is already part of the input cluster.

3.3. Coinbase Address Clustering

A coinbase transaction is the first transaction in a block. It is constructed by the miners to receive the mining reward along with the transaction fees in the block. A coinbase transaction is similar to a regular bitcoin transaction, the only difference being that the coinbase transaction contains a single empty input, called coinbase, and a set of output addresses specified by the miner to receive the reward. Miners often use the `scriptSig` field of the coinbase transaction to include a text string. This field unlocks input-related funds within a typical bitcoin transaction, typically by providing one's private key. However, for coinbase transactions, there is no need to provide a script to unlock funds, as these are generated by the blockchain and are transferred to the miner. Miners typically use this field to include the name of their mining pool, but it can be used to enter any text string. In the early days, when the bitcoin protocol was launched, individual miners mined blocks. As the technology developed, this phenomenon gradually disappeared, and individual miners were replaced by mining pools. In mining pools, miners collaborate to solve the proof-of-work problem. Once a viable solution is found, the mining pool is responsible for distributing the rewards of the mined block to all the miners who participated in the mining process in proportion to the computational power made available by each of them. As a result, more and more miners are opting for mining pools to reduce energy costs and obtain more stable revenue. Lewenberg et al. [19] analyzed the operational structure of bitcoin mining pools. Based on this work, heuristics have been implemented for address clustering, including one related to coinbase address clustering that was proposed by Zheng et al. [18]. Thus, the last heuristic used in the tool implementation concerns clustering all of the output addresses for a coinbase transaction.

Definition 3. *If a coinbase transaction contains two (or more) output addresses, they are all controlled by the same entity.*

Ordinarily, the output addresses a coinbase transaction unless it is a single miner belonging to a mining pool, as shown in Figure 3. These addresses will then be tasked with distributing the block rewards to the miners who participated in the pool.

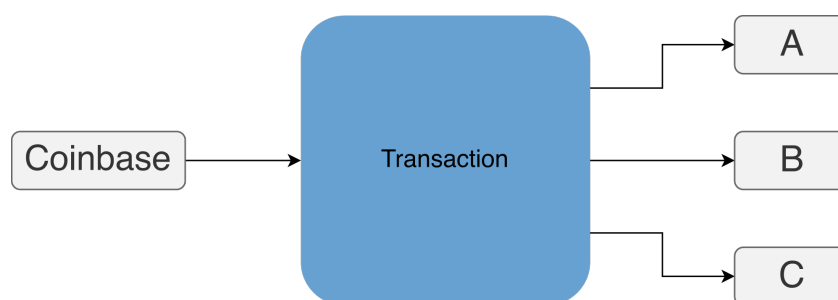


Figure 3. Coinbase address example: addresses A, B, and C belong to a mining pool.

The heuristic used in the proposed work adds an additional condition: if the number of output addresses in the coinbase transaction is greater than a certain threshold γ , then the heuristic is not applied to the transaction. This check is made because some mining pools, such as P2Pool and Eligius, distribute shares to miners directly in the coinbase transaction. It may occur, therefore, that the output addresses of a coinbase transaction refer to different users, and building a cluster with these elements would be incorrect. In the BACH tool, γ is assigned a value of 10, as this is considered a valid compromise based on experimental evidence.

Definition 4. *If the number of output addresses in a coinbase transaction is less than the γ threshold, then the addresses are all controlled by the same entity.*

In this way, many addresses that actually belong to the mining pool are not added to the relevant cluster. This also avoids clustering false positives together, which would contribute to the construction of untrue clusters.

3.4. Accuracy and Limitations

Each mining pool has its own internal distribution pattern, which changes frequently. The same considerations can be made for mixing services. For this reason, trying to identify a unique pattern for these services would be meaningless. Rather, heuristics with a certain degree of reliability are used and can be applied to all bitcoin services indiscriminately. Several studies have been conducted regarding the accuracy of the heuristics used for clustering bitcoin addresses. Gong Yanan and Chow [24] analyzed the error rates of some clustering heuristics. Clearly, the accuracy of heuristics does not depend solely on the heuristic itself, but other factors also contribute to determining it, such as the case of CoinJoins for multi-input clustering and the distribution patterns of some mining pools for coinbase clustering. Chang et al. [25] used the Gini impurity index to measure the homogeneity of the clusters found. To calculate it, information regarding the identity of each cluster must be added, which is not currently present in the BACH tool. Each cluster ideally should contain only addresses belonging to the same entity, but in reality, this is not always the case, and a cluster may happen to have addresses belonging to different entities. Therefore, the following formula is used to calculate cluster homogeneity:

$$Gini(f) = 1 - \sum_{i=1}^m f(i)^2 \quad (1)$$

where m is the number of different tags found within a given cluster, i is the index of each tag, and $f(i)$ is the fraction of bitcoin addresses in the cluster belonging to tag i . A tag corresponds to an entity that was previously found by off-chain information analysis. A pure cluster should contain only addresses belonging to a single entity, and its Gini index would equal 0. But when false positives are included in the cluster, the Gini index can increase to a maximum of 1.

4. BACH

BACH is logically composed of three different components:

- The database containing the blockchain data;
- The server that provides this data through the database;
- The client that displays the results.

The network architecture follows a client–server model with a database, for which the communication flow between the various nodes is as follows:

- The client application accepts input from the user and sends an HTTP request to the server;
- The server opens a connection with the database, performs the query, and receives the result;
- The server sends the result obtained to the client application that made the request;
- The client application displays the cluster data received from the server in a 3D graph.

(<https://github.com/semifredd0/Bach> (accessed on 10 september 2024)).

4.1. Database Construction

A script was developed to retrieve transaction data from the public blockchain and build a database with all addresses divided into clusters using the three heuristics described in the previous section. In addition, all the relationships between addresses created by individual transactions are stored and are then used to visualize the cluster using a 3D graph. All addresses found during the scan are stored within the database. In addition, information about the internal structure of the clusters is stored, i.e., the relationships that bind the various addresses within the cluster, which depend on the heuristics used.

The database schema is shown in Figure 4.

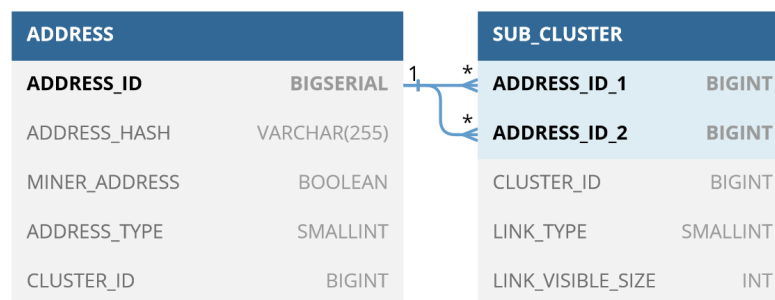


Figure 4. Schema of the database used by BACH.

The address table contains information regarding the addresses found and the clusters to which they belong; there is also a field to specify the type of address (Legacy, Pay To Script Hash, Native SegWit, or Taproot) and a field to indicate whether the address is a miner, i.e., it appears as an output for at least one coinbase transaction. The sub_cluster table, on the other hand, contains information on the relationships between different pairs of addresses within the same cluster; in addition, for each pair, the type of relationship is indicated, i.e., the heuristics used to construct it: this information is used in the 3D graph to highlight the different relationships using different colors.

After database construction, to speed up the reading operations in the API, the following indexes were defined:

- `create index hash_index on address(address_hash);`
- `create index subcluster_index on sub_cluster(address_id_1, address_id_2).`

Some additional solutions were adopted to improve the script execution performance and the speed of 3D graph rendering: All the hashes of the blocks to be parsed are inside a file in the project directory and are loaded into memory at the beginning of the script execution to optimize the speed of reading this data. The database connection is left open during the script's execution, so the queries are executed much faster. In addition, the connection is closed and reopened every 100 blocks to avoid losing data, and a backup is made. Replacing the Scanner object with the BufferedReader sped up the download operation and the mapping of blocks within objects, greatly improving the overall execution time.

4.2. Server Architecture

After building the database, REST APIs were implemented to allow external applications to access the data. The technology used was ExpressJS: a NodeJS library for API development. Specifically, endpoints were defined as those that receive GET-type requests via the HTTP protocol, query the database, and return data in JSON format based on the parameters passed to the request. The endpoints are defined as follows:

- `GET/{address}`: returns all addresses belonging to the address cluster passed as parameter.
- `GET/sub/{address}`: returns all links between addresses belonging to the cluster of the address passed as a parameter.
- `GET/info/{address}`: returns all the links of the address passed as a parameter.

These can then be used by the client application, i.e., the BACH tool, to access the data stored in the database.

4.3. Web Application

Finally, a web application was implemented to allow the data in the clusters to be visualized through a 3D graph and tabular structures. The technology used to implement the application is React: a front-end JavaScript library for creating user interfaces. The tool allows the user to set a bitcoin address in the search bar and display some information

about the address cluster. In particular, the tool allows the display of a table of all addresses belonging to the same cluster as the found address and a 3D graph of the cluster that is constructed using the heuristics presented above. In addition, by clicking on one of the nodes in the graph, it is possible to visualize in detail all the links of the latter to the other addresses in the cluster. A 3D force-directed graph component is used to construct the 3D graph. It allows the data structure of a graph to be represented in a three-dimensional space and uses ThreeJS/WebGL for 3D rendering. The tool's homepage provides more information regarding the cluster graph, such as the colors used for the nodes and arcs. The tool allows moving within the 3D graph—zooming in and changing angles—to better visualize certain nodes and links present in the cluster, as shown in Figure 5. In addition, clicking on a node within the graph will show all its links to other nodes and the types of heuristics used for linking, as shown in Figure 6.

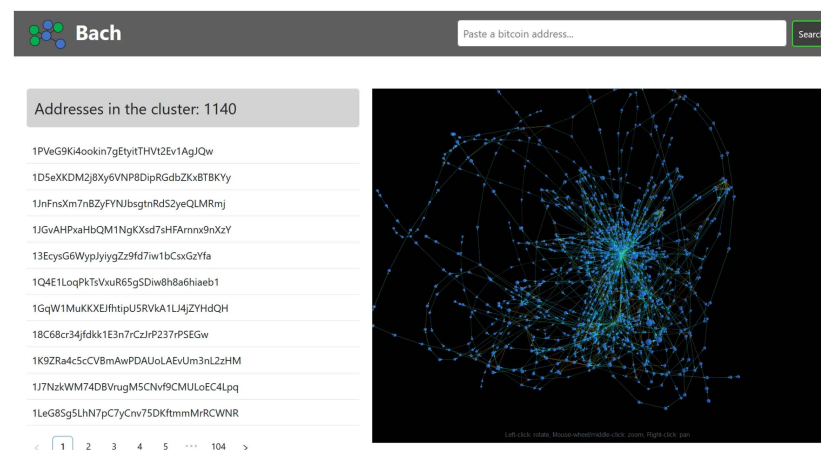


Figure 5. Listing all addresses within the same cluster and showing the cluster's 3D graph.

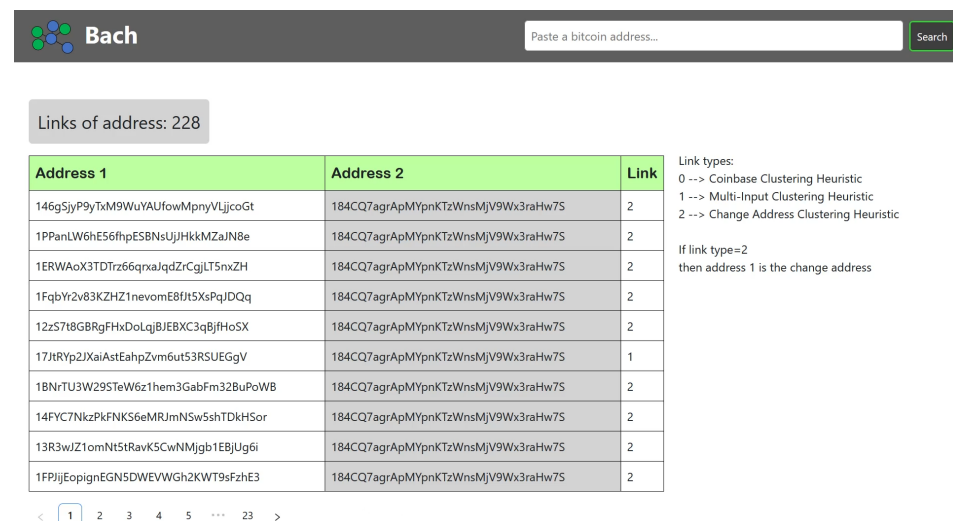


Figure 6. Seeing all of the related links by clicking on a node.

A negative element is the rendering speed of the graph, which on devices with small capacities could take several tens of seconds, especially if the cluster to be displayed is very large. To remedy this problem, the `link_visible_size` field in the `sub_cluster` table was introduced in the database; it avoids rendering all the links within a cluster. This allows even less powerful devices to view the entire cluster graph without experiencing major slowdowns. In the client code, it is possible to modify this field so that if a node contains more links than the set limit, they are not rendered within the graph. This aspect is useful in cases where there are fairly large groups of addresses in which each has a link to every

other address in the group: this is the case with groups that are formed using multi-input clustering heuristics; indeed, all input addresses for the same transaction have a link to all others. If the number of input addresses is N , then the relationships between them will be $N!$, i.e., a significantly large number that grows factorially with N . It is important to note that although these links are not shown in the graph, the nodes are still displayed correctly, as shown in Figures 7 and 8.

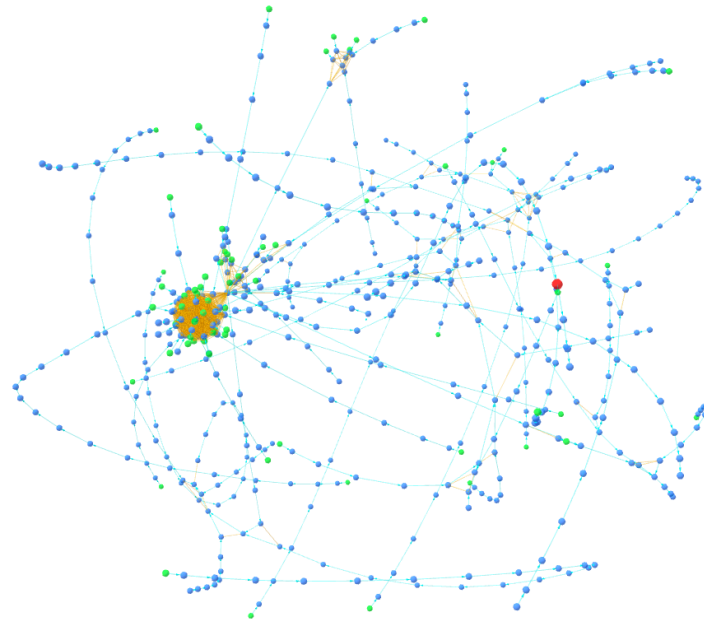


Figure 7. Viewing with no limitation on the display of links.

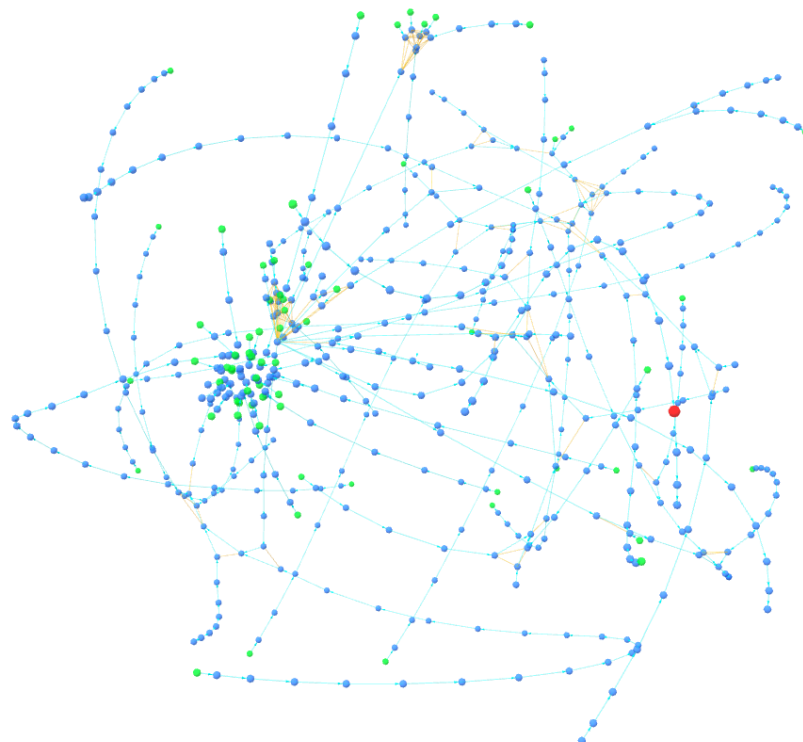


Figure 8. Viewing with active link display limit.

5. Experimentation

This section reports the experiments carried out to validate the functionality of BACH. The experiments were performed considering the first 130,000 blocks of the bitcoin blockchain. The script for building the database ran on a Windows 10 machine with 16 GB RAM and AMD Ryzen 7 4700U processor. The first 130,000 blockchain blocks were parsed over a runtime of about a week. The final size of the database is 6454 MB. After the first 100,000 blocks, execution slowed down considerably. This was due to the increasing number of transactions within the blocks and, more importantly, the increasing database size. In addition to reading and inserting operations, editing operations are also required on existing records, such as updating the ClusterID of an address group. Since some fields are changed frequently, it is impossible to define indexes on them to speed up reading.

5.1. Detection of Peeling Chains

A peeling chain is a transactional pattern that is used mainly by mixing services and allows large sums of money to be laundered through a long series of small transactions. This process is illustrated in Figure 9. Basically, the peeling chain starts with one address receiving a certain amount of money. This address then sends the money to two (or more) addresses: one still belongs to the transaction's sender and represents the change address. The change address repeats the peeling process until all the money runs out.

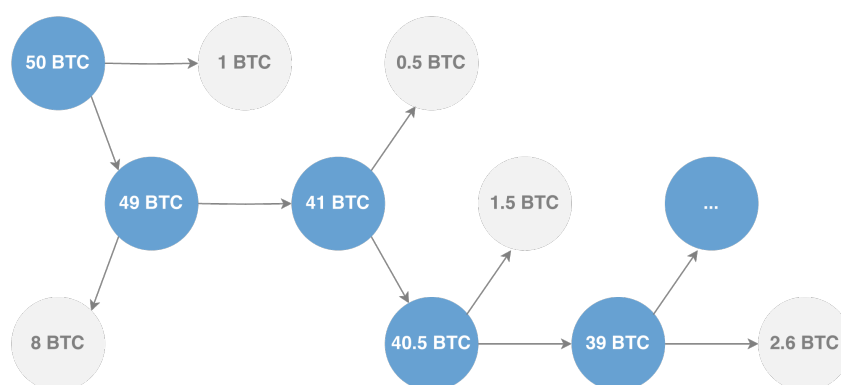


Figure 9. Peeling chain process. Each node in the peeling chain uses only two addresses for each transaction: one is the address where the peel is sent (in grey), and the other is the change address, where the remaining amount of money is sent (in blue). The address chain in blue represents the peeling chain.

Typically, the address to which peels are sent belongs to an exchange, where they are usually converted into fiat currency or other types of assets. Criminals typically use very long and complex peeling chains to prevent tracking of their funds. Note that this pattern does not have to be attributed to money laundering activities but can also be used by ordinary users who wish to maintain privacy and avoid being tracked by clustering tools. Through the BACH tool, some clusters were identified that could be the results of peeling chains. In fact, as can be seen in Figures 10 and 11, these clusters exhibit a spiral shape in which each cluster address represents the change address of another. The tool, in fact, thanks to the 3D visualization of the cluster graph, makes it easy to identify any patterns present within it. The clusters are very similar and are shown in the figure from the same angle. Each address is linked to another through change address heuristics, as seen by the light-blue-colored links between the various nodes. This type of structure often indicates the presence of a peeling chain.

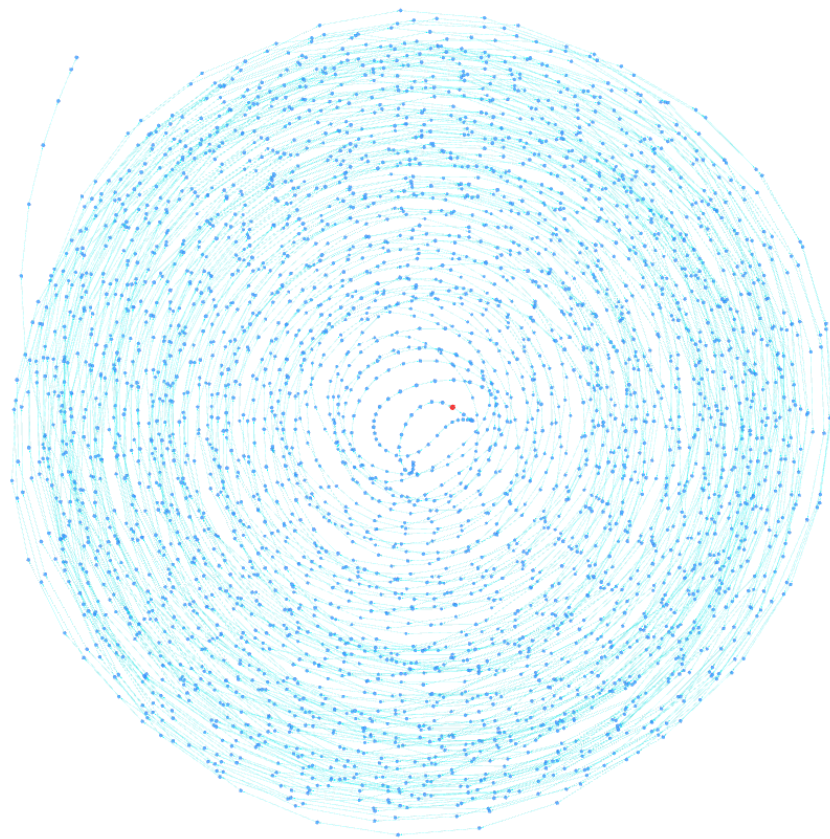


Figure 10. Cluster A graph.

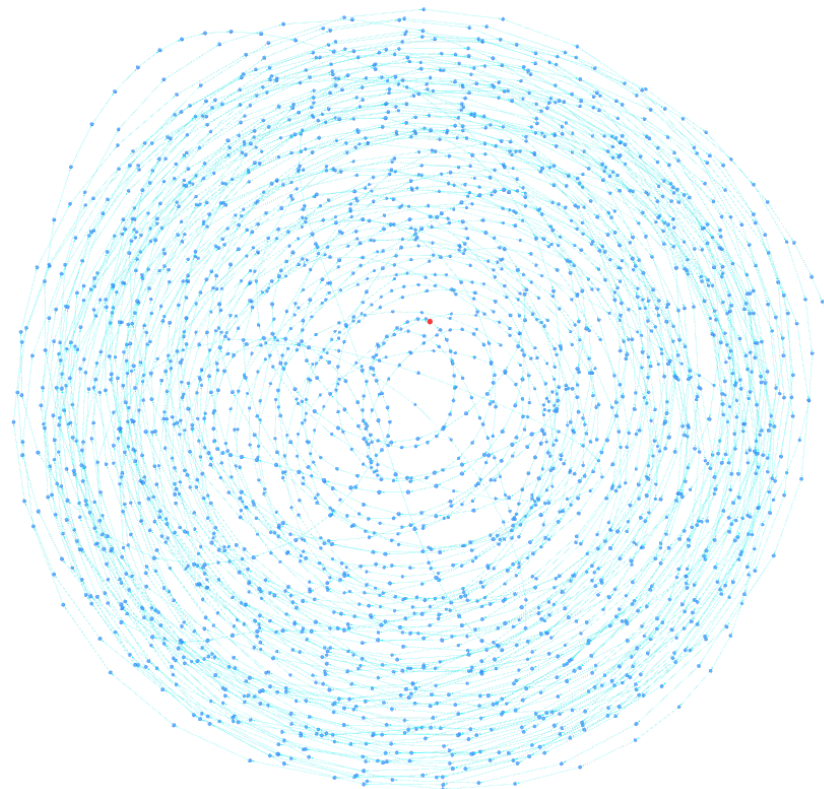


Figure 11. Cluster B graph.

Cluster A contains 3659 addresses, while Cluster B contains 2781 addresses. You can analyze them in detail with the BACH tool by entering the following bitcoin addresses:

- Cluster A: 162G6uzHJpmxsM3EQFDLzEYcmx1hxnJtRR;
- Cluster B: 1Lgne9nu4ZzVyfqarr2Mdp8JmhsB3amvA8.

The patterns are represented by a Sankey diagram: a special flow chart in which the width of the arrows is drawn proportionally to the amount of flow. Sankey diagrams, which were drawn using the online tool SankeyMatic, are very useful for visualizing the money flow between addresses. The spiral-shaped clusters identified by the BACH tool represent peeling chains. Each address in the chain, as it first appears in the blockchain, is identified as the change address of the transaction. This way, all addresses in the peeling chain are correctly placed in the same cluster. On the other hand, a limitation of the tool is that the addresses where peels are sent are not included in the cluster. The first pattern identified uses only one address to which the peel is sent and can be schematized as in Figure 12. Each node in the peeling chain always sends the peel to the same address, eventually containing the same amount of money as the address that starts the chain. This is the simplest configuration of a peeling chain. The other two patterns identified instead use multiple peeling addresses in a way that increases their complexity. Specifically, the second uses about ten peeling addresses, while the third uses hundreds. Figure 13 summarizes the process used by these patterns: in the example, only six peeling addresses were included, which will eventually contain the amount of money from the address that started the chain, but the process used by the patterns described is identical.



Figure 12. Peeling chain with single peeling address.

Analyzing the spiral clusters identified by the BACH tool, it was found that cluster B, shown in Figure 11, uses a peeling chain with multiple peeling addresses, similar to the pattern shown in Figure 13. The same cluster was investigated by Zhao et al. [17], who identified a peeling chain pattern by studying 166 mixing transactions. Specifically, BTC 125 were transferred from an address to eight peeling addresses. At the end of the process, the amount of money in the eight destination addresses matched the amount contained in the source address. BACH provides several useful information points regarding the cluster, but a limitation of the tool is that the peeling addresses are not placed in the same cluster as the peeling chain addresses.



Figure 13. Peeling chain with multiple peeling addresses.

5.2. Comparison to WalletExplorer

This section highlights the differences between the clusters found by BACH and those found instead by WalletExplorer. WalletExplorer (<https://www.walletexplorer.com/>) is a tool designed to cluster bitcoin addresses to help users explore and understand the relationships between different bitcoin wallets. Users can explore various wallets, view transaction histories, and see how wallets and transactions are interconnected through an easy-to-use interface on the WalletExplorer website. WalletExplorer provides the results in tabular form. BACH offers the same functionality as WalletExplorer, but it differs by implementing three heuristics for address grouping instead of just one. In addition, BACH allows the cluster graph to be displayed with all the relationships between addresses, which are not stored using WalletExplorer. To perform this type of analysis, the database was reconstructed using only the multi-input clustering heuristic, i.e., the only heuristic used by WalletExplorer for constructing clusters. In this way, the results obtained by the two tools after analyzing the same dataset will be compared, since WalletExplorer also analyzes transactions after the block at a height of 130,000. Next, some of the limitations of WalletExplorer are listed:

- Uses only the multi-input clustering heuristic, thus not aggregating clusters that have a high probability of belonging to the same entity;
- Does not allow visualization of relationships between addresses in the same cluster but merely displays them all in the same table;
- Because of the previous point, it is impossible to visualize the cluster's internal structure graphically since the relationships are not stored.

BACH aims to solve these problems by using three clustering heuristics and storing all the relationships between addresses for each analyzed transaction while indicating the heuristics used to find them. This information allows one to trace the internal structure of the cluster, which is illustrated in a 3D graph. The clusters were analyzed by reconstructing the WalletExplorer database with the same scheme used by BACH but by exclusively implementing the multi-input clustering heuristics. This method also allows a comparison

of the internal structures of the clusters obtained by the two tools. The following images show a comparison of the identified clusters.

Figure 14 displays a cluster obtained exclusively using the multi-input clustering heuristic; in fact, all its links are orange, i.e., the various nodes are related to each other only via multi-input relationships; the graphical visualization was obtained using BACH. Figure 15, on the other hand, shows the same cluster but constructed using the BACH tool: in fact, it is possible to visually notice the difference in the number of nodes and relationships between the two clusters. In addition, other types of links are also present, which indicate the use of the new clustering heuristics introduced by the tool. It is important to note that some small clusters obtained with the multi-input heuristics are not clustered together using WalletExplorer, while the BACH tool aggregates some of them to the main cluster using the other two heuristics. The following address can be searched using the BACH tool to analyze the cluster shown in the figure: `1wEmdvd75rGsTiR8myP1F9os8yPUySUKJ`. A further example is shown below, this time using the Slush Pool cluster [26]. The same considerations can be made here as well.

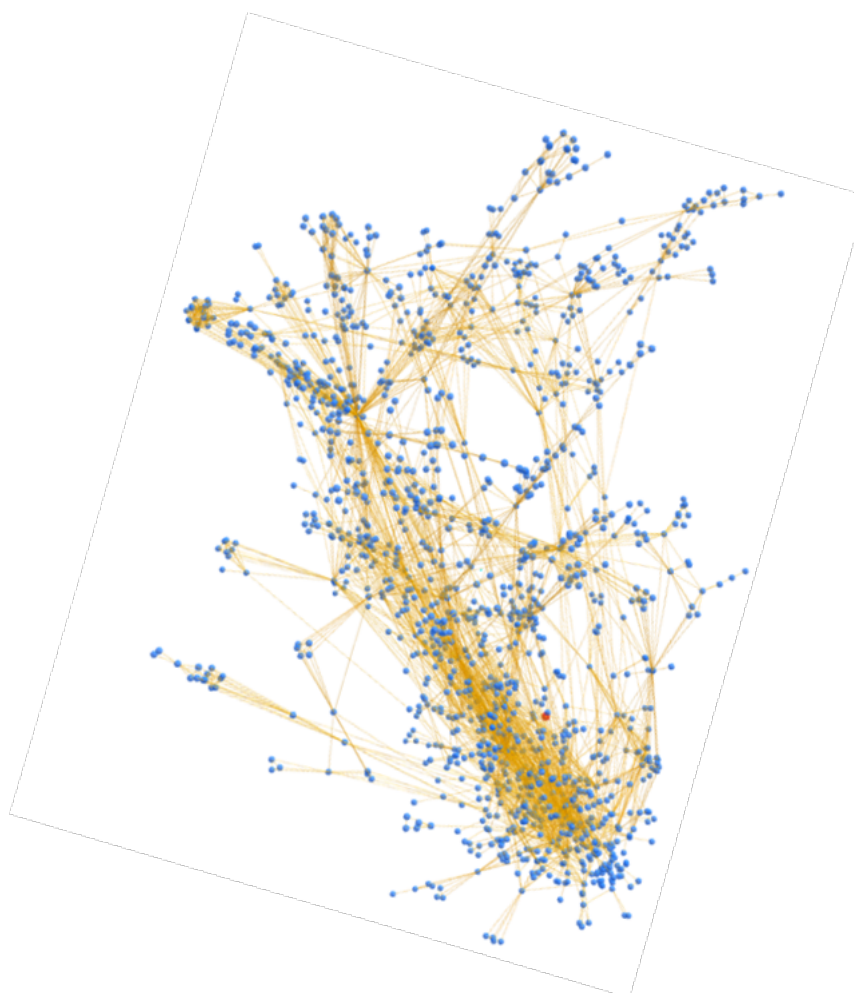


Figure 14. Cluster obtained using the BACH graphics engine but using only the heuristics implemented by WalletExplorer.

In Figure 16, the cluster contains only the nodes linked to the central address, while in Figure 17, it is possible to see how these addresses are also linked to others through new relationships, which in most cases are identified by change address clustering heuristics. Ultimately, this analysis shows how BACH favors an increase in address aggregation within clusters compared to WalletExplorer and introduces a novel feature to observe the clusters from a new point of view.



Figure 15. Cluster obtained using BACH.

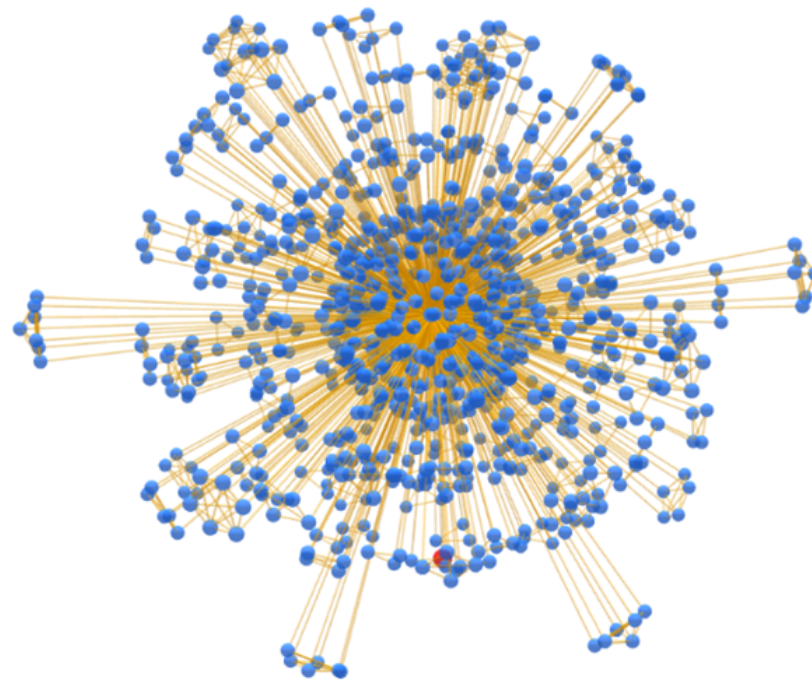


Figure 16. Cluster obtained using the BACH graphics engine but using only the heuristics implemented by WalletExplorer.



Figure 17. Cluster obtained using BACH.

Table 1 shows some information about the results obtained by the two tools.

Table 1. Comparison of the results obtained by the two tools.

Indicators	WalletExplorer	BACH
Number of total clusters	54,441	62,865
Number of total relations	3,597,427	4,315,813
Size of the largest cluster	10,084	25,377
Database size	4074 MB	6454 MB

In contrast to expectations, the number of total clusters turns out to be higher in the case of BACH, which is expected to contain a smaller number of clusters as it aims to aggregate smaller clusters into larger ones. This goal was indeed achieved by BACH, but the larger number of clusters identified can be attributed to the use of the new coinbase clustering heuristic, which identifies a large number of clusters by analyzing coinbase transactions. In fact, there are a total of 11,006 clusters consisting solely of miners, as many of the initial blocks of the blockchain contain multiple output addresses for coinbase transactions; as a result, many clusters are formed due to the coinbase clustering heuristic. By removing these clusters from the figure shown in the table, a total of 51,859 clusters are obtained. This is a lower number than that of WalletExplorer, which shows how the introduction of the two new heuristics improved the degree of address aggregation in the network. In addition, the table shows data on the number of total relationships, which is obviously higher in BACH's case, and the number of addresses from which the largest cluster is composed. Finally, the size of the databases containing the data used by the two tools is shown: in this case, most of the space is clearly occupied by information about the relationships within the various clusters. Thus, the database will be significantly larger in the case of BACH, which contains many more relationships than WalletExplorer. It is important to emphasize that the size given for the WalletExplorer database does not correspond to the actual size of the database used by the service since the database was reconstructed to carry out these analyses: in fact, WalletExplorer does not store the internal relationships for each cluster, greatly reducing its size.

Some interesting data regarding the sizes of the clusters found and the distributions of addresses for the various clusters are now shown. The two graphs in Figure 18 and Figure 19 show the sizes of the 100 largest clusters obtained by the BACH and WalletExplorer analyses, respectively.

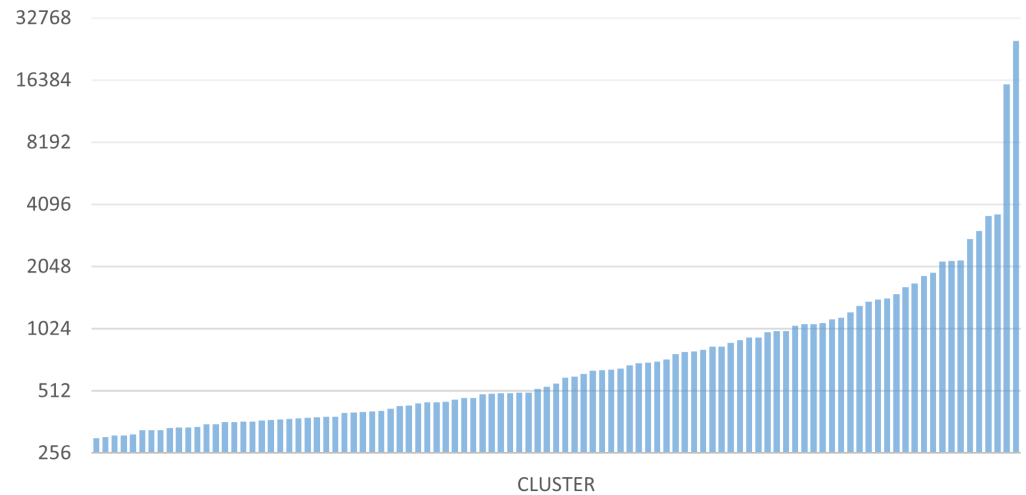


Figure 18. Size of the 100 largest clusters obtained using BACH.

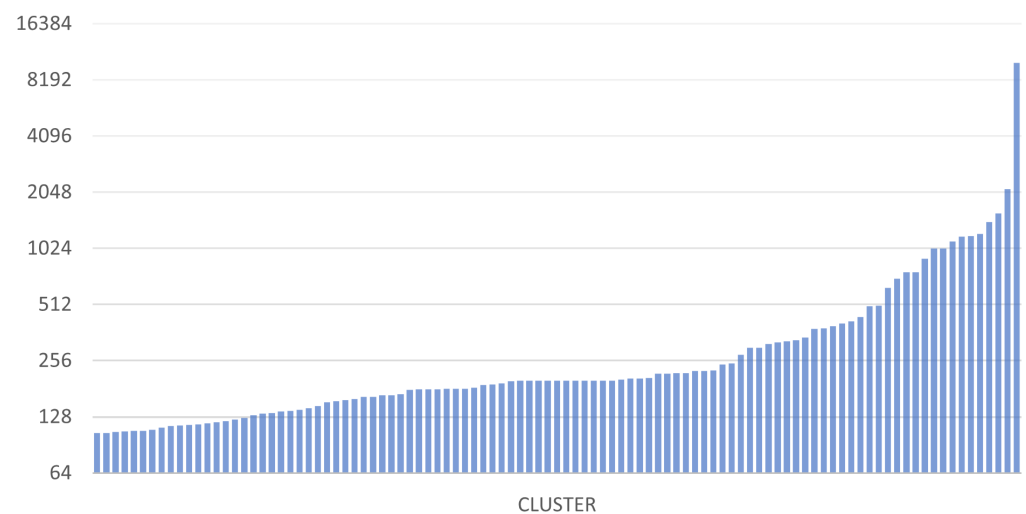


Figure 19. Size of the 100 largest clusters obtained using WalletExplorer.

These graphs show that the number of addresses per cluster increased evenly, improving the level of address aggregation. The average number of addresses per cluster using BACH also grew compared to WalletExplorer. This aspect can be seen even more clearly in the two graphs in Figures 20 and 21, in which the values on the y -axis indicate the size of the clusters, while those on the x -axis indicate the number of clusters with a given size.

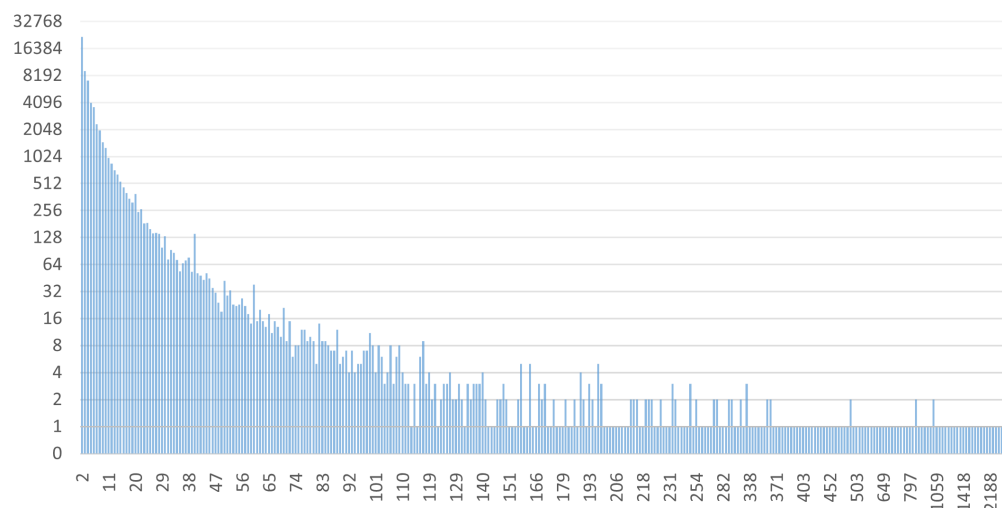


Figure 20. Distribution of addresses in clusters obtained using BACH.

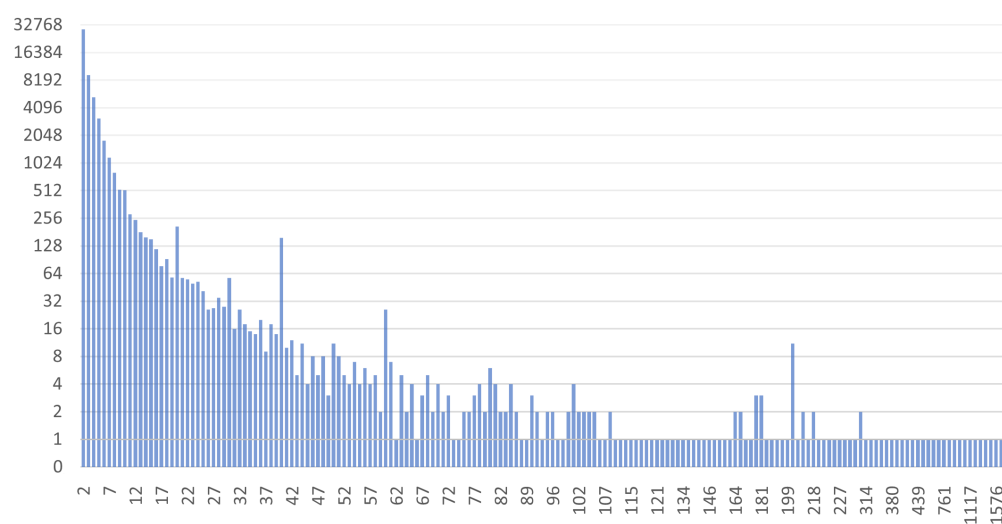


Figure 21. Distribution of addresses in clusters obtained using WalletExplorer.

In Figure 20, it is possible to observe how the curve settles around the value of 100, while in Figure 21, this happens around the value of 50. This finding indicates that BACH can detect many more clusters that are larger than those detected by WalletExplorer, for which most of the clusters found contain a maximum of about 50 addresses. The number of addresses analyzed is the same for both tools, but the distributions of these within the various clusters has changed dramatically. In fact, in the case of BACH, the number of addresses not belonging to any cluster decreased by about 60% compared to the WalletExplorer analysis. In terms of accuracy and efficiency, the results obtained from using both tools can be seen in the analyses above. The combined use of three heuristics greatly favors the level of cluster aggregation, highlighting the advantage of BACH over WalletExplorer. In terms of usability, both tools provide an interactive and easy-to-use graphical user interface. In the case of BACH, just pasting a bitcoin address into the search bar is enough to display its possible cluster, while in the case of WalletExplorer, the user can also search by TXID, wallet ID, or service name. In addition, WalletExplorer displays all the addresses in a cluster within a table without highlighting the relationships between them, whereas BACH allows the cluster to be visualized and explored graphically, thus facilitating the identification of any patterns visible only after reconstructing the structure based on the clustering heuristics used. Finally, the following are some of BACH's limitations compared to its counterpart WalletExplorer: BACH does not track the balance present at the various

addresses within clusters and the related transactions that led to the construction of that balance. In addition, WalletExplorer also adds service names for some clusters for which it is able to identify the name.

6. Conclusions

This work proposed BACH: a tool that can analyze bitcoin transactions and identify clusters of addresses that potentially belong to a single entity. Such a tool is more effective than those already known as it uses multiple combined heuristics to identify the address cluster. BACH works on bitcoin but is still extendable to other cryptocurrencies operating via blockchain. The operation of BACH is superior to other already known tools because it uses multiple heuristics simultaneously. Experiments showed that BACH is particularly effective at detecting transactional patterns, and that the clusters detected are more complete and substantial. Such a tool thus proves useful when deanonymizing transactions on the blockchain, particularly when illicit activity or money laundering is suspected. BACH sees genuine applicability in real-world contexts. One of the most interesting possible applications of BACH involves tracking transactions on illicit markets such as Silk Road. By clustering addresses, law enforcement agencies can identify patterns and track fund flows, leading (in combination with other techniques) eventually to the identification and arrest of key traders. Clustering heuristics can effectively group addresses associated with Silk Road and similar illicit markets. Using specific heuristics, it is possible to identify groups of addresses that are likely to be controlled by the same entity or involved in related transactions. As shown in Section 5, combining multiple heuristics increases the accuracy of bitcoin address clustering, thereby reducing the risk of false positives and increasing the ability to understand the dynamics underlying the operation of the bitcoin network. This method reveals links that would otherwise not be directly observable with a single heuristic, highlighting that the complexity of the relationships buried beneath the blockchain requires more sophisticated and integrated analytical approaches. Another scenario where BACH could be useful involves ransomware attacks. Ransomware operators often demand payments in bitcoin. BACH can help to group addresses associated with ransomware payments, facilitating the tracking of ransom payments and identifying wallets used by cybercriminals. Clustering ransomware payment addresses and their interactions with exchange services provides insights into attackers' cash-out methods, aiding in the development of countermeasures. However, limitations such as false positives, evasion techniques, scalability issues, and legal concerns must be addressed to optimize BACH's application and ensure its ethical use in the fight against crime. This paper contributes to the theory of blockchain analysis by proposing a new multi-heuristic clustering approach that provide more complete and accurate inferences than single-heuristic approaches. Graph-theoretic modeling of clusters in 3D enables advanced internal structure research, which has been relatively unexplored in the literature. The identification of peeling chain patterns, which are commonly used in malicious activities, adds to the theoretical understanding of transaction dynamics. Additionally, it forms the basis for future integration of machine learning to classify and detect similar illicit patterns automatically. Some limitations in real-world applications include:

- Clustering algorithms can sometimes incorrectly link unrelated addresses or fail to link related addresses, leading to false positives and false negatives, respectively. This can result in misidentification and wrongful suspicion of innocent parties. BACH aims to minimize false positives through the use of heuristics and accurate thresholds, but since it is a heuristic approach, perfect results cannot be achieved.
- Criminals continuously evolve their methods to evade detection. Techniques such as using multiple wallets, coin mixing services, and privacy-centric cryptocurrencies can reduce the effectiveness of BACH.
- The sheer volume of bitcoin transactions can pose scalability challenges for BACH. Efficiently processing and analyzing large datasets requires significant computational resources.

Some possible future developments are listed below:

- Researchers can update the algorithm by adding new clustering heuristics. These enhancements could improve the accuracy of address clustering and enable the identification of more complex transaction patterns.
- The integration of machine learning algorithms could be used to find common patterns within clusters and classify other similar entities that, despite following the same model, have not yet been classified. For example, the clusters of major bitcoin services (exchanges, mining pools, etc.) could be analyzed, and features could be extracted from the transactions. These large entities often execute a high volume of transactions.

Moreover, cluster graph modeling should not be understood merely as a graph-assisted visualization tool, as it opens the way for different perspectives. The literature mainly focuses on the composition of clusters and not on their internal structure. New studies could, therefore, expand the discipline through the application of the elements of graph theory to cluster composition since BACH provides the internal structure. Likewise, as the multi-heuristic approach generates large and complete clusters, techniques for refining these a posteriori could be explored based on the internal connections within the same. In addition to offering innovative graphical visualizations of clusters, BACH demonstrates improved address aggregation by effectively utilizing three different clustering heuristics.

Author Contributions: Conceptualization, F.V., S.G. and F.D.L.; methodology, M.C. (Matteo Costantin), S.G. and F.V.; software, F.V., F.D.L. and M.C. (Michele Caringella); validation, F.V., F.D.L. and S.G.; formal analysis, M.C. (Matteo Costantin), S.G. and F.V.; investigation, M.C. (Matteo Costantin), S.G. and F.V.; resources, F.V., M.C. (Michele Caringella) and F.D.L.; data curation, F.V., M.C. (Michele Caringella) and S.G.; writing—original draft preparation, M.C. (Matteo Costantin), S.G. and F.V.; writing—review and editing, M.C. (Matteo Costantin), S.G. and F.D.L.; visualization, F.V., S.G. and M.C. (Matteo Costantin); supervision, F.V., M.C. (Michele Caringella) and S.G.; project administration, F.V., M.C. (Michele Caringella) and F.D.L. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by Regione Puglia—Progetto ACROSS—AcCounting & payRoll Software cloud converSion—POR PUGLIA FESR 2014-2020—Grant number: THA48Y5.

Data Availability Statement: Data used in this work are public.

Acknowledgments: No form of artificial intelligence was used in the creation of the content of this project.

Conflicts of Interest: Author Michele Caringella, Francesco Violante and Francesco De Lucci were employed by the company Italtpaghe S.r.l. The remaining authors declare that the research was conducted in the absence of any commercial or financial relationships that could be construed as a potential conflict of interest.

References

1. Nakamoto, S. Bitcoin: A Peer-to-Peer Electronic Cash System. 2008. Available online: https://www.usssc.gov/sites/default/files/pdf/training/annual-national-training-seminar/2018/Emerging_Tech_Bitcoin_Crypto.pdf (accessed on 10 September 2024).
2. Raju, R.S.; Gurung, S.; Rai, P. An overview of 51% attack over Bitcoin network. In *Contemporary Issues in Communication, Cloud and Big Data Analytics: Proceedings of CCB 2020*; Springer: Berlin/Heidelberg, Germany, 2022; pp. 39–55.
3. Kaminsky, D. Some Thoughts on Bitcoin. Available online: <https://dankaminsky.com/2011/08/05/bo2k11/> (accessed on 25 June 2024).
4. Irwin, A.S.; Turner, A.B. Illicit Bitcoin transactions: Challenges in getting to the who, what, when and where. *J. Money Laund. Control* **2018**, *21*, 297–313. [[CrossRef](#)]
5. Meiklejohn, S.; Pomarole, M.; Jordan, G.; Levchenko, K.; McCoy, D.; Voelker, G.M.; Savage, S. A fistful of bitcoins: Characterizing payments among men with no names. In *Proceedings of the 2013 Internet Measurement Conference*, Barcelona, Spain, 23–25 October 2013; pp. 127–140.
6. Shojaeinasab, A.; Motamed, A.P.; Bahrak, B. Mixing detection on bitcoin transactions using statistical patterns. *IET Blockchain* **2023**, *3*, 136–148. [[CrossRef](#)]
7. Hong, Y.; Kwon, H.; Lee, J.; Hur, J. A practical de-mixing algorithm for bitcoin mixing services. In *Proceedings of the 2nd ACM Workshop on Blockchains, Cryptocurrencies, and Contracts*, Incheon, Republic of Korea, 4 June 2018; pp. 15–20.

8. Wu, J.; Liu, J.; Chen, W.; Huang, H.; Zheng, Z.; Zhang, Y. Detecting mixing services via mining bitcoin transaction network with hybrid motifs. *IEEE Trans. Syst. Man Cybern. Syst.* **2021**, *52*, 2237–2249. [[CrossRef](#)]
9. De Balthasar, T.; Hernandez-Castro, J. An analysis of bitcoin laundry services. In Proceedings of the Secure IT Systems: 22nd Nordic Conference, NordSec 2017, Tartu, Estonia, 8–10 November 2017; Proceedings 22; Springer: Berlin/Heidelberg, Germany, 2017; pp. 297–312.
10. Kokash, N. *An Introduction to Heuristic Algorithms*; Department of Informatics and Telecommunications: Trento, Italy, 2005; pp. 1–8.
11. Kinkeldey, C.; Fekete, J.D.; Isenberg, P. Bitconduite: Visualizing and analyzing activity on the bitcoin network. In Proceedings of the EuroVis 2017—Eurographics Conference on Visualization, Posters Track, Barcelona, Spain, 12–16 June 2017; p. 3.
12. Yue, X.; Shu, X.; Zhu, X.; Du, X.; Yu, Z.; Papadopoulos, D.; Liu, S. Bitextract: Interactive visualization for extracting bitcoin exchange intelligence. *IEEE Trans. Vis. Comput. Graph.* **2018**, *25*, 162–171. [[CrossRef](#)] [[PubMed](#)]
13. Tovanich, N.; Heulot, N.; Fekete, J.D.; Isenberg, P. Visualization of blockchain data: A systematic review. *IEEE Trans. Vis. Comput. Graph.* **2019**, *27*, 3135–3152. [[CrossRef](#)] [[PubMed](#)]
14. Androulaki, E.; Karame, G.O.; Roeschlin, M.; Scherer, T.; Capkun, S. Evaluating user privacy in bitcoin. In Proceedings of the Financial Cryptography and Data Security: 17th International Conference, FC 2013, Okinawa, Japan, 1–5 April 2013; Revised Selected Papers 17; Springer: Berlin/Heidelberg, Germany, 2013; pp. 34–51.
15. Ermilov, D.; Panov, M.; Yanovich, Y. Automatic bitcoin address clustering. In Proceedings of the 2017 16th IEEE International Conference on Machine Learning and Applications (ICMLA), Cancun, Mexico, 18–21 December 2017; pp. 461–466.
16. Zhang, Y.; Wang, J.; Luo, J. Heuristic-based address clustering in bitcoin. *IEEE Access* **2020**, *8*, 210582–210591. [[CrossRef](#)]
17. Zhao, Z.; Wang, J.; Shi, K.; Zhang, H. Improving Address Clustering in Bitcoin by Proposing Heuristics. *IEEE Trans. Netw. Serv. Manag.* **2022**, *19*, 3737–3749. [[CrossRef](#)]
18. Zheng, B.; Zhu, L.; Shen, M.; Du, X.; Guizani, M. Identifying the vulnerabilities of bitcoin anonymous mechanism based on address clustering. *Sci. China Inf. Sci.* **2020**, *63*, 1–15. [[CrossRef](#)]
19. Lewenberg, Y.; Bachrach, Y.; Sompolinsky, Y.; Zohar, A.; Rosenschein, J.S. Bitcoin mining pools: A cooperative game theoretic analysis. In Proceedings of the 2015 International Conference on Autonomous Agents and Multiagent Systems, Istanbul, Turkey, 4–8 May 2015; pp. 919–927.
20. Reid, F.; Harrigan, M. An Analysis of Anonymity in the Bitcoin System. In Proceedings of the 2011 IEEE Third International Conference on Privacy, Security, Risk and Trust and 2011 IEEE Third International Conference on Social Computing, Boston, MA, USA, 9–11 October 2011; pp. 1318–1326. [[CrossRef](#)]
21. Ron, D.; Shamir, A. Quantitative Analysis of the Full Bitcoin Transaction Graph. In *Financial Cryptography and Data Security*; Sadeghi, A.R., Ed.; Springer: Berlin/Heidelberg, Germany, 2013; pp. 6–24.
22. Spagnuolo, M.; Maggi, F.; Zanero, S. BitIodine: Extracting Intelligence from the Bitcoin Network. In *Financial Cryptography and Data Security*; Christin, N., Safavi-Naini, R., Eds.; Springer: Berlin/Heidelberg, Germany, 2014; pp. 457–468.
23. Maxwell, G. Coinjoin: Bitcoin Privacy for the Real World. Available online: <https://bitcointalk.org/?topic=279249> (accessed on 25 June 2024).
24. Gong, Y.; Chow, K.P.; Ting, H.F.; Yiu, S.M. Analyzing the error rates of bitcoin clustering heuristics. In Proceedings of the IFIP International Conference on Digital Forensics, Virtual, 3–5 January 2022; Springer: Berlin/Heidelberg, Germany, 2022; pp. 187–205.
25. Chang, T.H.; Svetinovic, D. Improving bitcoin ownership identification using transaction patterns analysis. *IEEE Trans. Syst. Man Cybern. Syst.* **2018**, *50*, 9–20. [[CrossRef](#)]
26. Hercog, U.; Povše, A. Taint analysis of the Bitcoin network. *arXiv* **2019**, arXiv:1907.01538.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.